

Optimization Methods for Machine Learning and Engineering

Lecture 6 – Automatic Differentiation

Julius Pfrommer



CC BY-SA 4.0

Agenda

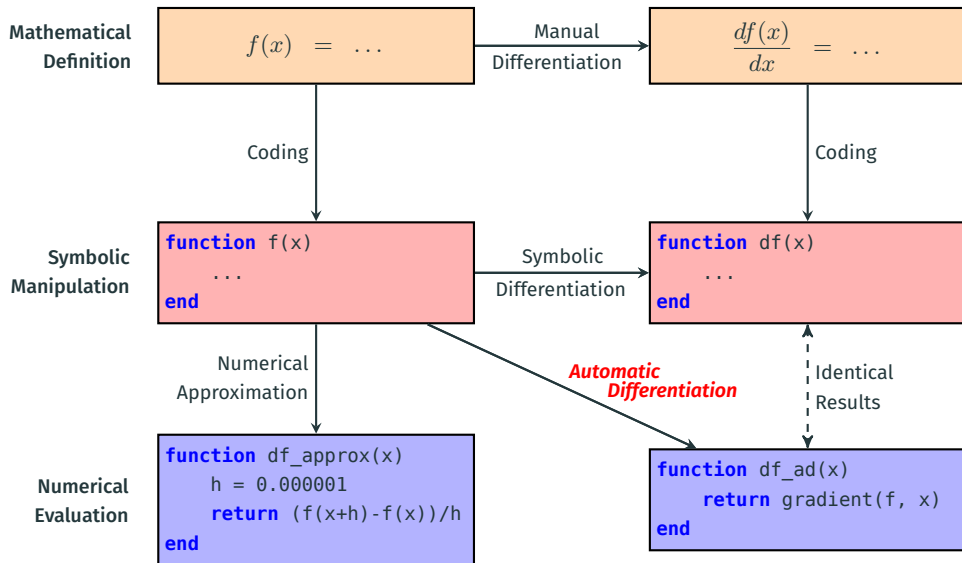
1. Automatic Differentiation

2. Space Mission Planning

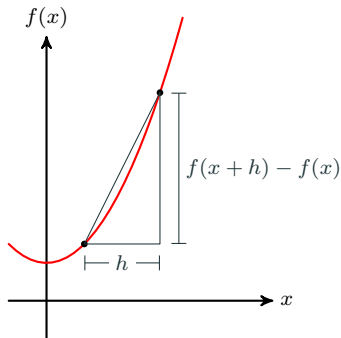
3. Artificial Neural Networks

Automatic Differentiation

Alternative Ways to Compute Derivatives

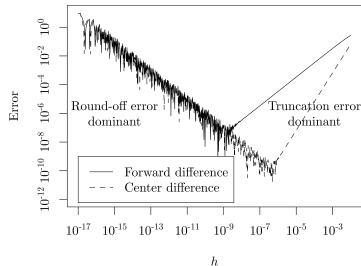


Numerical Approximation of Derivatives



$$e_{\text{forward}}(x, h) = \left| \frac{f(x+h) - f(x)}{h} - \frac{d}{dx}f(x) \right|$$
$$e_{\text{center}}(x, h) = \left| \frac{f(x+h) - f(x-h)}{2h} - \frac{d}{dx}f(x) \right|$$

Error of the Numerical Approximation [Baydin2015]



- The approximation is limited by the **precision of the IEEE-754 floating point numbers**
- For **large** h the error decreases linearly with decreasing h due to numerical imprecision
- For **small** h the error increases linearly with decreasing h due to round-off errors and overflow effects
- The center difference method reaches a smaller error

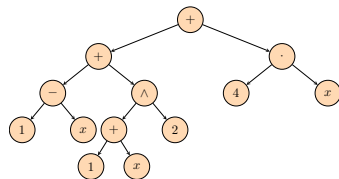
Finding Derivatives from Symbolic Representations

Symbolic Computation of Derivatives

- Computer Algebra Systems (CAS) **represent mathematical expressions symbolically**
- When an expression is a combination of elementary expression (with the aforementioned combination rules), then algorithms can **“mechanically” find the derivative** and further **simplify the result expression**
- This is easy enough for an exercise in a 1st semester course [SICP1985] or lazy-sunday code challenges (<https://nbviewer.jupyter.org/github/norvig/pytudes/blob/master/ipynb/Differentiation.ipynb>)

Symbolic Integration

- Symbolic Integration is **much harder!** Some expressions provably have **no closed-form antiderivative** (e.g. x^x , $\sin(x^2)$)
- The *Risch Algorithm* determines closed-form integrals if they exist – for the most important types of expressions – and goes over 100 pages [Risch1969]
- Some of the early-day CAS have survived until this day as Open Source (e.g. REDUCE [Hearn1967], MACSYMA [Martin1971], AXIOM [Griesmer1971])

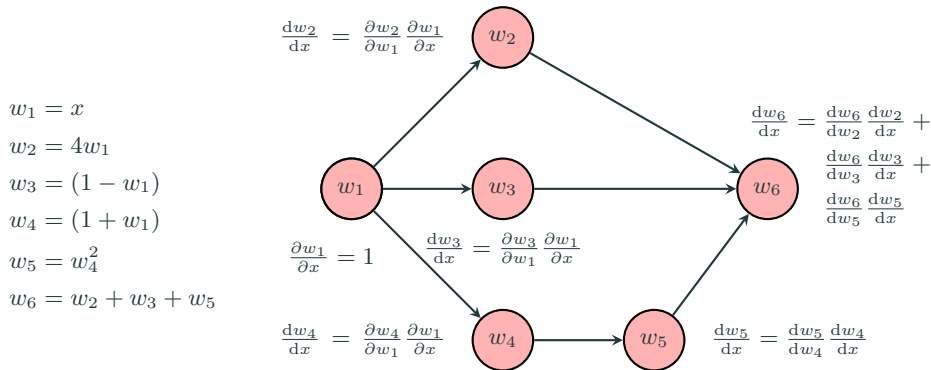


Expression tree for
 $(1 - x) + (1 + x)^2 + 4x$

The Evaluation Graph

The expression $f(x) = (1 - x) + (1 + x)^2 + 4x$ is decomposed into **nodes** w_i connected in a network.

We can follow how the **gradient “flows” in the network**.

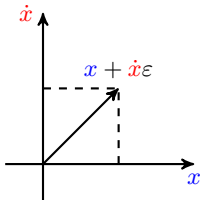


$$\frac{df}{dx} = \frac{dw_6}{dx} = \frac{\partial w_6}{\partial w_2} \frac{\partial w_2}{\partial w_1} \frac{\partial w_1}{\partial x} + \frac{\partial w_6}{\partial w_3} \frac{\partial w_3}{\partial w_1} \frac{\partial w_1}{\partial x} + \frac{\partial w_6}{\partial w_5} \frac{\partial w_5}{\partial w_4} \frac{\partial w_4}{\partial w_1} \frac{\partial w_1}{\partial x}$$

Dual numbers carry the derivative together with the undifferentiated answer (called the primal).

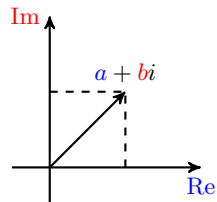
$$x = x + \dot{x}\varepsilon, \quad \text{where } x, \dot{x} \in \mathbb{R}, \varepsilon^2 = 0$$

Dual Numbers



- Two parts: primal x and derivative $\dot{x}$
- ε is an abstract number with $\varepsilon^2 = 0$

Complex Numbers



- Two parts: real part a and imaginary part b
- i is an abstract number with $i^2 = -1$

Differentiation Rules and the Algebra of Dual Numbers

1. Addition and Scalar Multiplication

$$\begin{aligned}(f + g)' &= f' + g' & \Leftrightarrow & \quad \mathbf{x} + \mathbf{y} = ((x + \dot{x}\varepsilon) + (y + \dot{y}\varepsilon)) = (x + y) + (\dot{x} + \dot{y})\varepsilon \\ (af)' &= af' & \Leftrightarrow & \quad a\mathbf{x} = a(x + \dot{x}\varepsilon) = ax + a\dot{x}\varepsilon\end{aligned}$$

2. Product rule

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x) \quad \Leftrightarrow \quad \mathbf{x}\mathbf{y} = (x + \dot{x}\varepsilon)(y + \dot{y}\varepsilon) = xy + (\dot{x}y + x\dot{y})\varepsilon$$

3. Power rule with natural numbers as the exponent r

$$(x^r)' = rx^{r-1} \quad \Leftrightarrow \quad \mathbf{x}^r = (x + \dot{x}\varepsilon)^r = \sum_{k=0}^r \binom{r}{k} x^{r-k} (\dot{x}\varepsilon)^k = x^r + rx^{r-1}\dot{x}\varepsilon$$

Equivalent results emerge from the differentiation rules and by the simple application of the algebraic rules of dual numbers.

Evaluating Functions on Dual Numbers

A differentiable function $f(x) : \mathbb{R} \rightarrow \mathbb{R}$ evaluated for a dual number x returns

$$f(x) = f(x + \dot{x}\varepsilon) = f(x) + f'(x)\dot{x}\varepsilon.$$

Proof: The full Taylor Expansion around an initial point x is an exact representation of f .

$$\tau_x(y) = f(x) + f'(x)(y - x) + \frac{1}{2}f''(x)(y - x)^2 + \dots$$

Plugging x into τ_x gives the result:

$$\begin{aligned} f(x) = \tau_x(x) &= \tau_x(x + \dot{x}\varepsilon) = f(x) + f'(x)(\dot{x}\varepsilon) + \underbrace{\frac{1}{2}f''(x)(\dot{x}\varepsilon)^2}_{0} + \dots \\ &= f(x) + f'(x)\dot{x}\varepsilon \end{aligned}$$

4. Chain rule

$$\begin{aligned} f(g(x))' &= f'(g(x))g'(x) \quad \Leftrightarrow \quad f(g(x)) = f(g(x + \dot{x}\varepsilon)) = f(g(x) + g'(x)\dot{x}\varepsilon) \\ &= f(g(x)) + f'(g(x))g'(x)\dot{x}\varepsilon \end{aligned}$$

AD Dual Numbers in Julia

```
import Base: +,/,*,-.,^,sin,cos,exp,log
```

```
struct D <: Number
    p::Float64 # primal
    d::Float64 # derivative
end
```

```
# Overloading the arithmetic rules
```

```
+(x::D,y::D) = D(x.p + y.p, x.d + y.d)
-(x::D,y::D) = D(x.p - y.p, x.d - y.d)
-(x::D)      = D(-x.p, -x.d)
*(x::D,y::D) = D(x.p * y.p, x.p*y.d + x.d*y.p)
/(x::D,y::D) = D(x.p/y.p, (y.p*x.d-x.p*y.d)/(y.p^2))
^(x::D,k::Float64) = D(x.p^k, k*x.p^(k-1)*x.d)
^(x::D,k::Integer) = ^(x, float(k))
```

```
# Some well-known functions
```

```
sin(x::D) = D(sin(x.p), cos(x.p) * x.d)
cos(x::D) = D(cos(x.p), -sin(x.p) * x.d)
exp(x::D) = D(exp(x.p), exp(x.p) * x.d)
log(x::D) = D(log(x.p), x.d / x.p)
```

```
import Base: convert,promote_rule,show
```

```
# Automatic conversion and pretty-printing
```

```
convert(::Type{D}, x::Real) = D(x, 0.0)
promote_rule(::Type{D}, ::Type{<:Real}) = D
show(io::IO,x::D) = print(io, x.p, " + ", x.d, "€")
```

```
# Examples
```

```
D(5.0, 1.0) + D(7.0, 0.0) # 12.0 + 1.0€
D(5.0, 1.0) * 5           # 25.0 + 5.0€
D(5.0, 1.0) * D(7.0, 0.0) # 35.0 + 7.0€
D(5.0, 1.0)^3             # 125.0 + 75.0€
```

```
# Function examples
```

```
sin(D(1.0, 1.0))          # 0.841 + 0.540€
exp(log(D(2.0, 1.5)))     # 2.0 + 1.5€
```

```
# Custom function examples
```

```
sq(x) = x * x
sq(D(2.0, 1.0))          # 4.0 + 4.0€
sq(sq(D(2.0, 1.0)))     # 16.0 + 32.0€
```

Differentiate all the Algorithms!

```
import Base: ==, isless, abs

# Overloading the control flow operators
==(x::D, y::D)      = x.p == y.p
isless(x::D, y::D)   = x.p < y.p
isless(x::D, y::Number) = x.p < y
isless(x::Number, y::D) = x < y.p
abs(x::D) = (x >= 0) ? x : -x

# Example for the Huber Loss.
# Notice the case distinction!
function huber(x, delta=1.0)
    if abs(x) < delta
        return 0.5 * x^2
    end
    return delta * (abs(x) - 0.5 * delta)
end

huber(D(-0.2, 1)) # 0.02 + -0.2ε
huber(D(5, 2))   # 4.5 + 2.0ε
huber(D(-5, 2))  # 4.5 + -2.0ε
```

```
# Derivative of functions with a scalar argument
function gradient(f, x::Number)
    w = D(x, 1.0) # First derivative dx / dx = 1
    return f(w).d
end

# Gradient of functions with a vector argument
function gradient(f, x::Vector{N}) where N <: Number
    nx = length(x)
    d = zeros(nx)
    for i=1:nx
        w = [(j==i) ? D(x[i], 1.0) : x[i] for j=1:nx]
        d[i] = f(w).d
    end
    return d
end

# Examples
gradient(huber, -5) # -1.0
norm_squared(x) = sum(x.^2)
gradient(norm_squared, [3, 4]) # [6.0, 8.0]
```


Forward and Reverse Mode

Consider the multivariate function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with the Jacobian Matrix $J_{\mathbf{f}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}$.

Mathematically, using the Dual Numbers corresponds to what is called **Forward Mode AD**. (The same could have been achieved by symbolic reasoning instead of Dual Numbers.)

When $\mathbf{f}$ is evaluated with $\dot{x}_i = 1$ for one of the input dimensions i , we retrieve **one column of the Jacobian**.

$$\begin{pmatrix} \frac{\partial f_1}{\partial x_i} \\ \vdots \\ \frac{\partial f_m}{\partial x_i} \end{pmatrix}$$

In **Reverse Mode AD**, we first **record a “tape” of the control flow**. Then we can compute the derivatives of the output with respect to the input **going backwards through the recorded control flow**.

One row of the Jacobian is computed for each pass of Reverse Mode AD.

$$\begin{pmatrix} \frac{\partial f_i}{\partial x_1} & \dots & \frac{\partial f_i}{\partial x_n} \end{pmatrix}$$

Reverse Mode AD is more efficient than the Forward Mode when $n \gg m$. E.g. neural networks, where the loss function has a scalar output and a many-dimensional input (possibly millions of network parameters).

Space Mission Planning

Save Mark Watney!

Consider the following scenario: An **astronaut has stranded on Mars**. We want to send a rocket to pick him up and **bring him back to earth**. He can lift off from Mars (but get no further), so a **fly-by of Mars is sufficient** to pick him up.

The rocket has to be cheap and does not carry excess fuel into orbit. Hence it is put on a **ballistic trajectory to Mars and back**: After the initial launch the rocket is only accelerated by the gravitation of the sun and the planets.

Your mission (should you choose to accept it) is to design the course of the rocket through the solar system. For a ballistic trajectory this amounts to selecting the exact time, direction and speed at which the rocket leaves earth.



Theatrical Release Posters Copyright by 20th Century Fox

The scenario is loosely based on the science fiction novel THE MARTIAN and its movie adaptation.

The movie HIDDEN FIGURES showcases actual mission planning at NASA and the early use of digital computers.

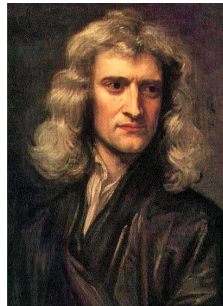
The n -Body Problem

The **n -Body Problem** is to describe the dynamics of a system of n massive objects interacting with each other by gravitation. With the effects of general relativity not yet taken into account.

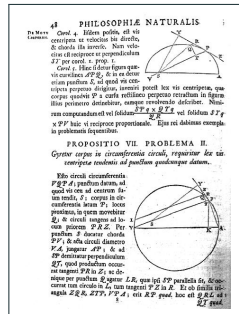
Describes the movements of the planets in our solar system around the sun – including the interactions between the planets.

In 1887 King Oscar II of Sweden announced a prize for anyone who could find a solution to the n -Body Problem. The prize was awarded to Poincaré in 1889 (even though he did not solve the problem).

While there exist analytic solutions for some configurations with $n > 2$, in general, **the n -Body Problems can only be solved by numerical integration (simulation).**



Isaac Newton
(1642 – 1726/27)



Isaac Newton. *Philosophiæ Naturalis Principia Mathematica*.
Royal Society, 1687

Leading up to Newton

- Copernicus (1543): Earth is not the center of the universe
- Kepler (1609): Planets have elliptic orbit around the sun
- Galileo (1638): Acceleration by gravitation

The n -Body Problem

Denote the **set of bodies** B with $|B| = n$. Each body $i \in B$ is described by its **point mass** m_i , position $\mathbf{p}_i$, speed $\dot{\mathbf{p}}_i$ and acceleration $\ddot{\mathbf{p}}_i$. The mass, initial position and speed are known.

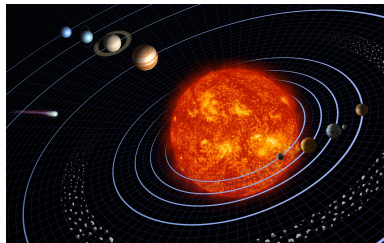
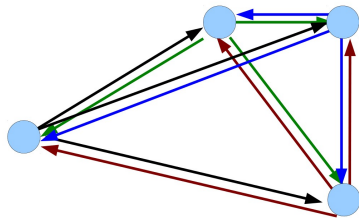
The **acceleration** $\ddot{\mathbf{p}}_i$ depends on the position of the other bodies j and their respective mass m_j creating **gravitational pull**:

$$\underbrace{m_i \ddot{\mathbf{p}}_i}_{\text{Acceleration}} = \underbrace{\mathbf{f}_i}_{\text{Force}} = \sum_{j \in B \setminus \{i\}} \frac{\gamma m_i m_j}{d_{ij}^2} \mathbf{e}_{ij}$$

where $d_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_2$ is the **Euclidean distance** between bodies i and j and $\mathbf{e}_{ij} = \frac{\mathbf{p}_j - \mathbf{p}_i}{d_{ij}}$ the normalized (length 1) direction of the distance vector.

The **gravitational constant** $\gamma = 6.67430 \cdot 10^{-11} \frac{\text{m}^3}{\text{kg s}^2}$ was first determined by [Cavendish1798].

If you want to go deep on **Classical Mechanics** (coming from Computer Science), go read [SICM].

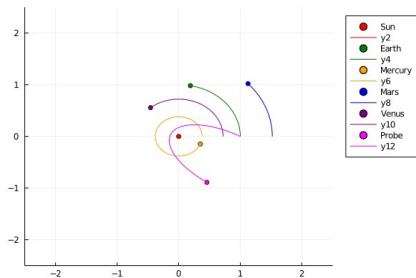


Numerical Simulation of the n -Body Problem

Similar to the previous lecture on Model Predictive Control, we **discretize the time domain** with steps of duration Δt and use the **Verlet Method** to simulate forward between time steps t and $t + 1$.

$$\dot{\mathbf{p}}_i^{(t+1)} = \dot{\mathbf{p}}_i^{(t)} + \ddot{\mathbf{p}}_i^{(t)} \Delta t$$

$$\mathbf{p}_i^{(t+1)} = \mathbf{p}_i^{(t)} + \dot{\mathbf{p}}_i^{(t)} \Delta t + \frac{1}{2} \ddot{\mathbf{p}}_i^{(t)} \Delta t^2$$



Body	Weight in kg	Avg. Distance to Sun in AU	Orbital Speed in $\frac{\text{km}}{\text{s}}$
Sun	1.989×10^{30}	0	0
Mercury	3.285×10^{23}	0.39	47.9
Venus	4.867×10^{24}	0.72	35.0
Earth	5.972×10^{24}	1.0	29.8
Mars	6.390×10^{23}	1.52	24.1

n-Body Simulation in Julia

```
AU      = 149_597_870_700    # in m
gamma   = 6.6743 * 10^(-11) # in m^3 kg^-1 s^-2
```

```
mutable struct Body{T}
    pos::Vector{T}
    speed::Vector{T}
    mass::Float64
end
```

```
# 2-Norm -> Euclidean Distance
norm2(x) = sum([xi^2 for xi in x])^0.5
```

```
# Bodies: Sun, Mercury, Venus, Earth, Mars
# m: Masses of planets in kg
# d: Initial distance to the sun in m
# s: Initial speed in m/s
m = [1.99e30, 3.28e23, 4.87e24, 5.97e24, 6.39e23]
d = [0.00, 0.38, 0.72, 1.00, 1.52] * AU
s = [0.00, 47.9, 35.0, 29.8, 24.1] * 1000
```

```
B = [Body([d[i], 0.0], [0.0, s[i]], m[i])
      for i=1:length(m)]
```

```
function simulate_step!(B::Vector{Body{T}}, Δt) where T
    n = length(B)           # number of bodies
    dims = length(B[1].pos) # length of position vector

    # Compute the acceleration
    acc = zeros{T, dims, n}
    for i = 1:n
        for j = setdiff(1:n, i) # skip i itself
            dij = norm2(B[i].pos - B[j].pos)
            acc[:,i] += gamma * B[j].mass *
                        (B[j].pos - B[i].pos) / (dij^3)
        end
    end

    # Update position and speed
    for i = 1:n
        B[i].pos += B[i].speed * Δt + 0.5 * acc[:,i] * Δt^2
        B[i].speed += acc[:,i] * Δt
    end
end
```

Optimizing the Rocket Trajectory

```
function trajectory_loss(theta::Vector{T}) where T
    # Initialize the planets
    B = [Body{T}([d[i], 0.0], [0.0, s[i]], m[i])
          for i=1:length(m)]

    # Rocket at ca. earth's position and speed plus theta
    push!(B, Body{T}(B[4].pos + ones(2) * 0.01*AU,
                     B[4].speed + theta, 5.0 * 10^4))

    # Simulate the planets and the ballistic trajectory
    dist_mars = zeros(T, tmax)
    dist_earth = zeros(T, tmax)
    for t=1:tmax
        simulate_step!(B, 10*3600) # 10h time steps
        dist_mars[t] = norm2(B[5].pos - B[6].pos)
        dist_earth[t] = norm2(B[4].pos - B[6].pos)
    end

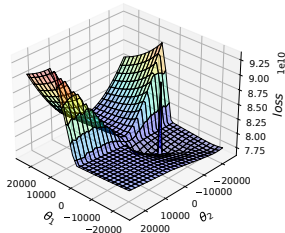
    # First to Mars, then back
    min_mars, idx = findmin(dist_mars)
    min_earth = minimum(dist_earth[idx:end])
    return min_mars + min_earth
end
```

Objective / Loss Function

For an initial rocket speed vector θ , the objective function (loss) penalizes the minimum distance to mars and the minimum distance to earth – after having been at Mars.

AD lets us differentiate the n -body simulation and therefore the loss function.

$$l(\theta) = \min_{t_{\max} > l > k > 0} \|p_{\text{mars}}^{(k)} - p_{\text{rocket}}^{(k)}\|_2 + \|p_{\text{earth}}^{(l)} - p_{\text{rocket}}^{(l)}\|_2$$



Artificial Neural Networks

Neurons, Synapses and Hebbian Learning

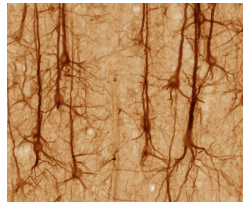
In the late 19th century, researchers Golgi and Ramón y Cajal discover techniques to **stain tissue samples** and describe the microscopic structure of the brain. Their findings are popularized by [Waldeyer1891] who also invents the term **Neuron**.

Neurons are connected via **synapses**. When a neuron is **sufficiently excited** through incoming signals (chemical / electrical), then **the neuron will fire** and signal further neurons [Sherrington1906].

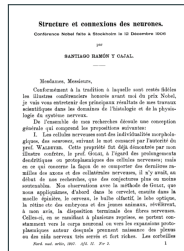
Hebbian Learning describes the mechanism for synaptic plasticity [Hebb1949]. Neurons that are active at the same time create synapses between them. Often-used synapses are strengthened (given more weight).

- “Cells that fire together, wire together.”
- “Any two cells or systems of cells that are repeatedly active at the same time will tend to become ‘associated,’ so that activity in one facilitates activity in the other.”

Stained Tissue Sample showing Neurons and Synapses



Nobel Lecture by Ramón y Cajal



Early Artificial Neural Networks

[McCulloch1943] describe networks of neurons with **binary excitations x and inhibitions y** and an activation threshold θ .

$$f_{\theta}(x, y) = \begin{cases} 0 & \text{if } \sum_i y_i > 0, \\ 1 & \text{if } \sum_i y_i = 0 \wedge \sum_j x_j > \theta, \\ 0 & \text{else} \end{cases}$$

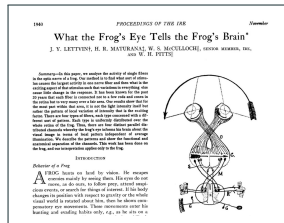
[Rosenblatt1962] invents the **Perceptron**. The activation rule for each neuron is simpler and includes **weights for the activation**.

$$f_{\theta}(x) = \begin{cases} 1 & \text{if } \mathbf{w}^{\top} \mathbf{x} + b > 0, \\ 0 & \text{else} \end{cases}$$

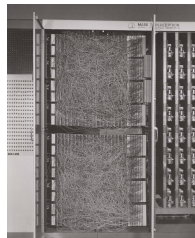
with $\theta = (\mathbf{w}, b)$ the parameters of the neuron.

[Minsky1969] shows that single-layer perceptrons cannot solve seemingly simple classification tasks (e.g. the XOR problem). They mention multi-layer perceptrons, but are pessimistic.

Nature as Inspiration [Lettvin1959]



Mark I Perceptron at the Cornell Aeronautical Laboratory (1957)



From Hebbian Learning to Backpropagation

A **non-differentiable activation function** makes training difficult. The **Sigmoid Function** $\sigma(z) = \frac{1}{1+e^{-z}}$ **is differentiable** and with appropriate scaling can approximate a step function. We write $\sigma(z)$ for the element-wise application to a vector $\sigma_i(z_i)$.

The network on the right-hand side has **fully connected layers**. With sigmoid activation it represents the mathematical function

$$m_{\theta}(x) = \sigma(\mathbf{W}_2 [\sigma(\mathbf{W}_1 x + \mathbf{b}_1)] + \mathbf{b}_2)$$

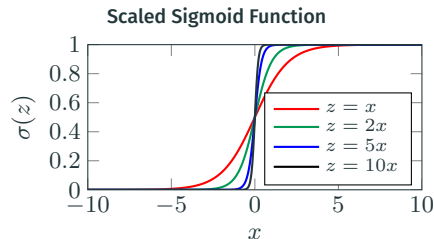
with model parameters $\theta = (\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2)$.

For the input-output data $D = \{(x_1, y_1), \dots\}$ let some **loss function**

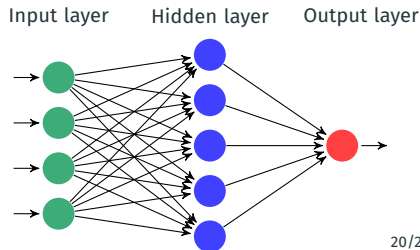
$$l_{\theta}(D) = \sum_j \|m_{\theta}(x_j) - y_j\|_2$$

Backpropagation is the idea to compute the gradient $\nabla_{\theta}[l_{\theta}(D)]$ across all network layers to minimize the loss [Rumelhart1986].

Backpropagation is “just” Automatic Differentiation!



A simple neural network
with fully connected layers



Stochastic Gradient Descent (SGD)

The loss functions of Neural Networks are generally not convex. They contain **many local minima and saddle-points**. Standard **Gradient Descent quickly “gets stuck” in local minima**.

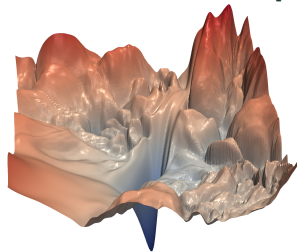
Second-order methods (Newton) are not helpful either:

- The Newton Method seeks saddle-points, here we want to escape them.
- Large number of parameters, cannot efficiently compute the Hessian!

Methods to find better solutions **and to avoid overfitting**:

1. **Stochastic Gradient Descent**: Vary the loss function between training iterations by randomly selecting subsets of D (minibatches) [Robbins1951; Bottou2004]
2. Modified gradient steps, e.g. based on some notion of momentum [Nesterov1983; Sutskever2013; Kingma2015]
3. Dropout (randomly disable neurons) [Srivastava2014]

Loss Function of a Neural Network [Li2018]



```
1: procedure SGD( $m, D, \theta, \eta, \beta, k_{\max}$ )
2:   for  $k = 1, \dots, k_{\max}$  do
3:      $D_k \leftarrow \bigcup_{i \in \text{rand}(|D|, \beta)} \{(\mathbf{x}_i, \mathbf{y}_i)\}$ 
4:      $\theta \leftarrow \theta - \eta \nabla_{\theta} [m_{\theta}(D_k)]$ 
5:   end for
6: end procedure
```

For a fixed learning rate η , batch-size β and $k_{\max}$ training iterations.

Using Neural Networks for OCR

Examples from the MNIST data set



In the 80s, early **convolutional neural networks** were used for the recognition of hand-written zip codes for the US Postal Service. The resulting MNIST data set is a canonical machine learning benchmark. Today MNIST is **essentially solved**.

Much more goes into a full-featured OCR engine. See the description of the Open Source project TESSERACT [Smith2007].

You will build a neural network that solves MNIST from scratch (only using Automatic Differentiation) in the exercises.



Yann Le Cun (*1960)

Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324

Training a Neural Network with Automatic Differentiation

```
sigmoid(z) = 1.0 / (1.0 + exp(-z))
```

```
function softmax(z)
    res = exp.(z .- maximum(z))
    return res / sum(res)
end
```

```
# A neural network with three fully connected layers
# theta: The network parameters
# im: A 28x28 matrix with the MNIST sample
# Output: A 10-vector with classification probabilities
```

```
function classify_mnist(theta, im)
    W1 = reshape(theta[1:100352], (128,28*28))
    b1 = theta[100353:100480]
    layer1(x) = sigmoid.(W1 * x + b1)

    W2 = reshape(theta[100481:108672], (64,128))
    b2 = theta[108673:108736]
    layer2(x) = sigmoid.(W2 * x + b2)

    W3 = reshape(theta[108737:109376], (10,64))
    b3 = theta[109377:109386]
    layer3(x) = softmax(W3 * x + b3)

    return layer3(layer2(layer1(vec(im))))
end
```

```
using MLDatasets
train_x, train_y = MNIST.traindata()
test_x, test_y = MNIST.testdata()
```

```
training_cycles = 10^5; learning_rate = 0.01; batchsize = 20
```

```
function loss_minibatch(theta)
    loss = 0.0
    for i=1:batchsize
        n = rand(1:length(train_y))
        res = classify_mnist(theta, train_x[:,n])
        loss += -log(res[train_y[n]+1]) # cross-entropy loss
    end
    return loss
end
```

```
# Backprop = Automatic Differentiation
grad_loss_minibatch(theta) = gradient(loss_minibatch, theta)
```

```
theta = rand(109386) # Initialize with random parameters
```

```
function train_mnist!(theta)
    for i = 1:training_cycles
        theta -= learning_rate * grad_loss_minibatch(theta)
    end
end
```

Summary of what you learned today

- **Automatic Differentiation (AD)**, a way to get derivatives “without doing the math”.
- **Dual Numbers** and operator overloading (in programming languages) to “derive” any numerical algorithm. (Be careful at the “edge” of case distinctions in the control flow!)
- Using AD for the optimization even of highly non-convex functions, using additional “bags of tricks”:
 - **Domain Grid Scanning + Gradient Descent:** In a **physics-based simulation (for space mission planning)**, AD is used to get the derivative of the objective function. Then use starting points from a regular grid and continue with Gradient Descent (or similar) to refine the solutions.
 - **Stochastic Gradient Descent:** Using AD for **backpropagation in an artificial neural network**, the loss function is randomly changed between training runs to escape saddle points and local minima.

That's it for today.

See you next week for Lecture 7 on the
Vector Spaces

- [SICP1985] Harold Abelson and Gerald Jay Sussman. *Structure and Interpretation of Computer Programs*. The MIT Press, 1996.
- [Bottou2004] Léon Bottou. “Stochastic Learning”. In: *Advanced Lectures on Machine Learning*. Ed. by Olivier Bousquet and Ulrike von Luxburg. Lecture Notes in Artificial Intelligence, LNAI 3176. Berlin: Springer Verlag, 2004, pp. 146–168. URL: <http://leon.bottou.org/papers/bottou-mlss-2004>.
- [Baydin2015] Atilim Gunes Baydin, Barak A. Pearlmutter, and Alexey Andreyevich Radul. “Automatic differentiation in machine learning: a survey”. In: *CoRR abs/1502.05767* (2015). arXiv: 1502.05767. URL: <http://arxiv.org/abs/1502.05767>.
- [Cavendish1798] Henry Cavendish. “Experiments to determine the density of the earth”. In: *Philosophical Transactions of the Royal Society of London* 88 (1798), pp. 469–526.
- [Griesmer1971] James H Griesmer and Richard D Jenks. “SCRATCHPAD/1: An interactive facility for symbolic mathematics”. In: *Proceedings of the second ACM symposium on Symbolic and algebraic manipulation*. 1971, pp. 42–58.

- [Hearn1967] Anthony C Hearn. “REDUCE: A user-oriented interactive system for algebraic simplification”. In: *Symposium on Interactive Systems for Experimental Applied Mathematics: Proceedings of the Association for Computing Machinery Inc. Symposium*. 1967, pp. 79–90.
- [Hebb1949] Donald Olding Hebb. *The organization of behavior: a neuropsychological theory*. J. Wiley, 1949.
- [Kingma2015] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the 3rd International Conference for Learning Representations*. 2015.
- [LeCun1998] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [Lettvin1959] Jerome Y Lettvin et al. “What the frog’s eye tells the frog’s brain”. In: *Proceedings of the IRE* 47.11 (1959), pp. 1940–1951.
- [Li2018] Hao Li et al. “Visualizing the loss landscape of neural nets”. In: *Advances in neural information processing systems*. 2018, pp. 6389–6399.
- [Martin1971] William A Martin and Richard J Fateman. “The MACSYMA system”. In: *Proceedings of the second ACM symposium on Symbolic and algebraic manipulation*. 1971, pp. 59–75.

- [McCulloch1943] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133.
- [Minsky1969] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press, 1969.
- [Nesterov1983] Yuri Nesterov. “A method of solving a convex programming problem with convergence rate $O(1/k^2)$ ”. In: *Sov. Math. Dokl.* Vol. 27. 2. 1983.
- [Newton1687] Isaac Newton. *Philosophiae Naturalis Principia Mathematica*. Royal Society, 1687.
- [Rumelhart1986] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536.
- [Risch1969] Robert H Risch. “The problem of integration in finite terms”. In: *Transactions of the American Mathematical Society* 139 (1969), pp. 167–189.
- [Robbins1951] Herbert Robbins and Sutton Monro. “A stochastic approximation method”. In: *The annals of mathematical statistics* (1951), pp. 400–407.
- [Rosenblatt1962] Frank Rosenblatt. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan Books, 1962.

- [Sherrington1906] Charles Scott Sherrington. *The Integrative Action of the Nervous System*. Yale University Press, 1906.
- [Smith2007] Ray Smith. “An overview of the Tesseract OCR engine”. In: *Ninth international conference on document analysis and recognition (ICDAR 2007)*. Vol. 2. IEEE. 2007, pp. 629–633.
- [Srivastava2014] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.
- [Sutskever2013] Ilya Sutskever et al. “On the importance of initialization and momentum in deep learning”. In: *International conference on machine learning*. 2013, pp. 1139–1147.
- [SICM] Gerald Jay Sussman and Jack Wisdom. *Structure and Interpretation of Classical Mechanics*. The MIT Press, 2001.
- [Waldeyer1891] Wilhelm von Waldeyer-Hartz. *Ueber einige neuere Forschungen im Gebeite der Anatomie des Centralnervensystems*. G. Thieme, 1891.
- [Wengert1964] Robert Edwin Wengert. “A simple automatic derivative evaluation program”. In: *Communications of the ACM* 7.8 (1964), pp. 463–464.

- Abloomfi - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=31159176>
- Dennis Nilsson, CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=3455682>
- Harman Smith and Laura Generosa (nee Berwin), graphic artists and contractors to NASA's Jet Propulsion Laboratory, with Pluto removed by User:Frokor - Based on Image:Solar_sys.jpg, with Pluto removed. Copied from http://en.wikipedia.org/wiki/Image:Solar_sys8.jpg, Public Domain, <https://commons.wikimedia.org/w/index.php?curid=1535590>
- <http://web.pd.astro.it/mapelli/Nbody1.pdf>